

Fine-Tuning and Mobilint MLA100 (ARIES) INT8 Compilation — Procedure & Commands

YOLOv11s on Argoverse-HD | RTX 5090 (FP32 training) + Mobilint MLA100 / ARIES (INT8 compile)

Self-contained record of the EXP-FT-LOCAL pipeline. All commands are the ones actually executed; model/artifact hashes are SHA-256 prefixes of the produced files.

0. Scope and key design decision

Goal: take the COCO-pretrained YOLOv11s, **fine-tune** it on Argoverse-HD, and produce an **INT8 model for the Mobilint MLA100 (ARIES) NPU**, then compare the INT8 quantization structure of the COCO model vs. the fine-tuned model.

Why a local paired compile (not the vendor blob). The paper's main NPU model is a vendor-published *.mxq* blob (HuggingFace mobilint/YOLO11s). A verification step showed the local compiler (qbcompiler 1.1.2) does **not** reproduce that vendor blob to within tolerance (per-size sAP Δ up to 0.008 > 0.003). Therefore this procedure compiles **both** the COCO and the fine-tuned model with the **same local recipe** so the comparison is internally controlled (recipe held fixed; only the weights differ). These INT8 numbers are an internal COCO-vs-FT comparison, not directly comparable to the vendor-blob Table 1.

1. Environments (two virtualenvs)

Training/inference and compilation use **separate** Python environments because the Mobilint compiler's native module requires a different PyTorch C++ ABI.

(a) Training / inference venv (.venv): torch 2.12.0+cu130, onnxruntime-gpu 1.20.1, ultralytics 8.4.56.

(b) Compiler venv (.venv_qbc): qbcompiler 1.1.2 (+aries2). Its native MMC module is built with the **C++11 ABI** and needs a cxx11-ABI PyTorch with CUDA (Blackwell sm_120) support — torch 2.12 (ABI mismatch) and torch 2.4 (no cxx11) both fail; **torch 2.8.0+cu128 (cxx11=True)** is the working combination.

Build the compiler venv (final working sequence):

Verified: torch 2.8.0+cu128 (cxx11=True), onnxruntime 1.20.1, qbcompiler 1.1.2; qbcompiler.mmc imports and CUDA runs on the RTX 5090.

2. Step 1 — Dataset preparation (Argoverse-HD → YOLO labels)

Argoverse-HD train annotations (COCO-style, 8 categories) are converted to Ultralytics YOLO labels. Labels are written in the **COCO-80 class space** (AHD id → COCO id mapping [0, 1, 2, 3, 5, 7, 9, 11]) so the fine-tuned model keeps the 80-class head and the existing evaluation harness (COCO→AHD mapping) is reused unchanged. Images are symlinked; a dataset YAML is written.

Resulting dataset YAML (key fields):

3. Step 2 — Fine-tuning (RTX 5090, FP32 PyTorch)

Ultralytics 8.4.56 defaults, fixed knobs only: 50 epochs, imgsz 640 (= inference resolution), seed 0, deterministic. Initial weights = the public COCO-pretrained yolo11s.pt. Run detached (survives disconnection).

Optimizer was Ultralytics auto (resolved to MuSGD, lr0=0.01). Outcome: 50 epochs completed; the **best checkpoint = epoch 1** (Ultralytics' default best-by-val-mAP; mAP50-95 peaked 0.236 at epoch 1 and declined afterwards). Best weights kept as-is (no cherry-picking): ft_runs/ft_yolo11s/weights/best.pt.

4. Step 3 — Export to ONNX (input to the compiler)

The compiler consumes ONNX. Both the COCO-pretrained and the fine-tuned weights are exported at 640×640, opset 13, static shapes, FP32.

5. Step 4 — Calibration set for INT8 quantization

INT8 quantization needs representative calibration inputs. A single fixed set of **200 Argoverse-HD train frames** (uniform sampling) is used for **both** compiles, so the calibration is identical across COCO and FT (only the weights differ).

6. Step 5 — Compile to MLA100 / ARIES INT8 (.mxq) with qbcompiler

Compilation uses the high-level `mxq_compile()` API of qbcompiler (product = ARIES / MLA100). Recipe held fixed across models:

- **config_preset = "yolo_640"**: YOLO 640×640 letterbox preprocessing (pad 114), uint8 input (the NPU does normalization), 3 image channels.
- **inference_scheme = "global8"**: all-8-core mode, used for the single-stream (Table-1-style) evaluation. A "single"-mode build (1 core/instance, up to 8 concurrent) is also produced for the N=4 multi-stream test.
- **backend = "onnx", device = "gpu"** (calibration runs on the host GPU).

Each compile took ~30–33 s and produced a ~10.7 MB .mxq (parse → quantize/calibrate → ARIES GlobalCluster build → export & verify).

7. Step 6 — Post-compile verification

- **Determinism / bit-identity**: each .mxq run on 20 frames twice → box/score diff = 0.
- **Operating point**: single-stream evaluation at threads=4, frame-skip ≈ 0.
- **FP32 gate before compiling**: fine-tuned vs COCO offline mAP (small +0.045, medium +0.069) confirmed fine-tuning raised accuracy before INT8 conversion.

8. Artifacts (SHA-256 prefix)

File	role	sha256[:16]
yolo11s.pt	COCO-pretrained init (Ultralytics v8.2.100)	85a76fe86dd8afe3
ft_yolo11s/weights/best.pt	fine-tuned FP32 (epoch 1 best)	4c4b342a6d09de67
yolo11s.onnx	COCO ONNX (640, opset13)	5ffc807bb21ba5f5
best.onnx	fine-tuned ONNX (640, opset13)	8e189e0252604e8f
qbc_coco_traincalib_global8.mxq	COCO INT8 (global8)	f987aca54e64c363
qbc_ft_traincalib_global8.mxq	fine-tuned INT8 (global8)	ea091a9dc2485a79
qbc_ft_traincalib_single.mxq	fine-tuned INT8 (single)	c0520594743655e4

Scripts: `ft_prep_dataset.py`, `ft_train_driver.sh`, `ft_dual_compile.py`, `ft_stretch_vlm.py` (all under `accv_experiments/scripts/`).
Detector COCO-pretrained weights: Ultralytics GitHub assets release. NPU compiler: qbcompiler 1.1.2 (+aries2), Mobilint ARIES / MLA100.